

Trace-Driven Proactive Autoscaling for Cluster Workloads via Multi-Horizon Forecasting

Jinchun Liu^{1*} and Haoxun Li¹

¹Hainan International College, Communication University of China, No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing, 100024, China.

*Corresponding author: Jinchun Liu
E-mail: ljc20041122@qq.com
Tel.: +86 18010695863
Fax: N/A

Abstract

Reactive autoscaling is easy to deploy but often responds too late. This study evaluates a reproducible predictive alternative on public Alibaba traces using aggregate, machine, transfer, and broad service-level evidence. Leakage-free forecasting shows that linear regression is the strongest average point baseline on the aggregate series (MAE 3.514), while **UA-MSTCN-Lite** still supplies usable **P90** coverage of 0.920, 0.867, and 0.868 for the 1-, 5-, and 10-minute horizons, respectively. A 12-machine cohort and zero-shot transfer confirm that simple linear structure remains hard to beat for point prediction. A broad audited service cohort of 139 **app.du** groups changes the service-level headline: linear regression becomes the strongest average point forecaster (MAE 32.571), while predictive control raises mean SLA violation from 0.017 to 0.031 and scaling actions from 249.0 to 399.4 versus reactive control; over-provisioning falls from 56.31 to 53.68 but is not stable under service bootstrap. Public traces therefore support reproducible predictive-autoscaling evaluation, multi-granularity evidence, and service-level boundary mapping, not a blanket superiority claim.

Keywords: cluster computing, workload forecasting, proactive autoscaling, Alibaba trace, resource management

1 Introduction

Large production clusters use elastic resource control to balance service quality and resource efficiency. Many stacks still rely on reactive threshold rules because they are simple and explainable [1–3], but such rules often scale out after demand has already surged and scale in only after

long idle periods. This creates a recurring trade-off between service-level violations and persistent over-provisioning.

Predictive autoscaling addresses this delay, but two gaps remain. Many studies evaluate forecasting and control separately, so it is unclear how lower prediction error changes policy trade-offs. Many strong cluster-management results also depend on private traces or platform-specific

deployments, limiting reproducibility and comparability.

Prior work on host-load prediction [4], container autoscaling [5], and uncertainty-aware workload forecasting on Alibaba and Google traces [6] already links forecasting, autoscaling, and resource management. The remaining need is a joint evaluation of prediction quality, control behavior, and validity limits.

This paper asks a narrower question: can public cluster traces support a reproducible predictive capacity-control evaluation that shows when decision trade-offs improve relative to reactive rules? The study uses Alibaba `cluster-trace-v2018`, whose documentation reports about 4000 machines over 8 days [7, 8]. Prior Alibaba analyses report diverse workloads and nontrivial resource-efficiency structure [9–11], making the trace suitable for this evaluation.

The pipeline routes multi-horizon forecasts into an autoscaling simulator with headroom, cooldown, and safety-margin constraints. This keeps the controller interpretable while allowing the forecasting backbone to be replaced without changing the evaluation route.

The evidence spans an aggregate series, a 12-machine high-coverage cohort, a cross-machine transfer benchmark, and a rule-based service cohort extracted from official `container_meta` and a full mirrored pass over `container_usage`. This structure tests whether aggregate forecasting and control patterns survive at machine and application-level units.

The implementation remains conservative. `UA-MSTCN-Lite` is a lightweight uncertainty-aware surrogate rather than the final deep `UA-MSTCN` architecture, and the service cohort is a broad rule-based benchmark rather than a full service census. The contribution is therefore evaluative: a reproducible public-trace benchmark and boundary-mapping study, not a claim that the current controller or surrogate is uniformly better.

This paper makes four evaluation-oriented contributions:

1. It frames predictive cluster capacity control as a reproducible public-trace evaluation problem.
2. It provides a pipeline from Alibaba trace acquisition to aggregate-, machine-, and service-level preprocessing, forecasting, and policy simulation.

3. It reports one consistent evidence surface across aggregate forecasting, machine-level benchmarking, cross-machine transfer, and service-level heterogeneity under one consistent protocol.
4. It maps the boundary of the current predictive route: linear baselines remain strong for point prediction, quantiles remain useful for control, and broad service evidence does not validate the shared global predictive policy as a service-level improvement route.

All preprocessing scripts, model training code, and the predictive-control simulator are publicly released at <https://github.com/Jinchun-Liu/paper-workbench> to guarantee exact reproduction of every reported number.

The remainder of the paper is organized as follows. Section 2 reviews adjacent work on workload characterization, forecasting, autoscaling, and generalization. Section 3 formalizes the forecasting and control problem. Section 4 describes the dataset routes, preprocessing, models, and evaluation protocol. Section 5 reports the forecasting and policy results. Section 6 discusses validity threats, and Section 7 concludes the paper.

2 Related Work

This section organizes prior work around the three layers needed for a trace-driven predictive-autoscaling study: workload characterization, forecasting, and control. The goal is not to claim that forecasting or autoscaling are new topics, but to identify where existing studies leave a reproducible public-trace gap between prediction quality and capacity-control behavior.

2.1 Representative precedent studies

Representative prior studies already span host-load prediction [4], container autoscaling and metric selection [5], service-granular microservice resource management [12, 13], uncertainty-aware workload forecasting with transfer learning [6], and recent predictive autoscaling frameworks for cloud environments [14]. Together these studies connect forecasting quality, autoscaling behavior, and realistic cluster workloads while emphasizing

different layers of that space. Table 1 summarizes how the present study differs in data source, control linkage, and evidence surface.

2.2 Trace-driven workload characterization

Public cluster traces are essential because they allow researchers to evaluate scheduling and resource-management ideas under realistic temporal variation. Google cluster traces played this role for earlier large-scale studies [15]. Alibaba later released several trace families that exposed co-located datacenter workloads, batch scheduling behavior, microservice metrics, and GPU-cluster behavior [7]. Follow-up studies characterized co-located workloads in Alibaba cloud datacenters [9], examined resource-efficiency bottlenecks [10], and analyzed large-scale microservice dependencies [11]. Recent cloud-evaluation work has also emphasized reproducible simulation and policy testing against realistic workloads and traces [16]. These studies provide strong descriptive or evaluative evidence, but they do not by themselves close the loop between forecasting and operational scaling decisions.

2.3 Forecasting for cluster and cloud workloads

Forecasting methods for systems workloads range from classical time-series models to neural sequence models. Empirical cloud-resource prediction has been studied with statistical and machine-learning methods for more than a decade [4, 17, 18]. Proactive provisioning studies then moved toward workload classification, multi-step forecasting, and explicit control relevance [19–21]. Sequence-learning models such as LSTM [22, 23] and more integrated deep-learning approaches [24] later became common choices for multistep forecasting. Probabilistic or quantile-aware forecasting is especially relevant for control problems because capacity decisions should respond not only to central tendency but also to uncertainty [6, 25, 26]. The practical challenge is not simply to achieve lower forecast error, but to expose decision-useful uncertainty without making the pipeline too heavy to reproduce.

2.4 Predictive autoscaling and uncertainty-aware control

Autoscaling has a long literature in cloud and grid computing, spanning threshold rules, control-theoretic policies, queuing-based methods, and predictive approaches [1, 27]. Container-era systems added new operational layers, including orchestration concerns [2, 3], proactive traffic-aware scaling [28], and Docker-specific elasticity mechanisms [29]. Resource-reservation studies also show that lightweight heuristic policies remain operationally relevant in public-cloud settings, especially when IoT-era demand variability makes over-reservation and SLA risk difficult to balance [30]. More recent service-granular studies examine resource management and cost-aware autoscaling directly at the microservice level [12, 13], while framework-oriented studies continue to integrate predictive models into end-to-end cloud autoscaling stacks and report operational gains under their target environments [14]. The common appeal of predictive policies is that they can scale before demand surges, but the common risk is that forecast noise can trigger unstable actions. Uncertainty-aware cloud studies therefore remain relevant because they make safety margins explicit [6, 31]. For this reason, a forecasting paper that never evaluates control outcomes, or a control paper that treats forecasts as opaque oracles, leaves an important systems gap unresolved.

2.5 Transfer learning and generalization

Recent workload-prediction papers increasingly discuss generalization across workloads and deployment contexts. This includes transfer-learning surveys [32], time-series transfer frameworks [33], and cloud-specific transfer-learning models [34, 35]. These studies make clear that generalization is already a substantive concern rather than an optional extension. That context matters for how the present study frames cross-machine transfer evidence and remaining limitations.

2.6 Position of the present study

The present study is positioned as an end-to-end public-trace evaluation rather than as a new autoscaling framework. Existing public-trace

Table 1 Representative prior studies on cluster forecasting and autoscaling.

Paper	Data	Forecast	Policy link	Relevance to the present study
Dinda and O'Hallaron [4]	Cluster host traces	Yes	No	Establishes early forecasting baselines at the cluster-host level.
Casalicchio [5]	Container experiments	Limited	Yes	Shows that metric choice and autoscaling behavior need to be evaluated together.
Rossi et al. [6]	Google and Alibaba traces	Yes	Service-level analysis	Demonstrates the relevance of uncertainty-aware forecasting and transfer analysis on public traces.
Present study	Alibaba public trace	Yes	Yes	Couples forecasting, transfer, and trace-driven control under one reproducible public-trace pipeline.

studies are often descriptive, forecasting studies frequently stop before policy evaluation, and predictive autoscaling studies often depend on private data or platform-specific implementations or emphasize positive framework-specific deployment results [14]. The gap addressed here is a reproducible, trace-driven forecasting-to-control benchmark that can be evaluated end to end using public artifacts while making mixed and negative service-level findings explicit rather than hiding them.

3 Problem Formulation

Let $e \in \mathcal{E}$ denote the managed entity under study. Depending on the experiment, e may represent the aggregate cluster, an individual machine, or an application-level service identified by Alibaba's `app_du`. For every entity, the time index is discretized to one-minute intervals. The observed demand signal is denoted by $x_{e,t} \in \mathbb{R}_{\geq 0}$.

Entity-level forecasting objective.

Given a context window of length L , the forecasting layer receives

$$\mathbf{x}_{e,t-L+1:t} = \{x_{e,t-L+1}, x_{e,t-L+2}, \dots, x_{e,t}\},$$

and predicts future demand at horizons $\mathcal{H} = \{1, 5, 10\}$ minutes. For each $h \in \mathcal{H}$, the model produces at least two quantiles,

$$\hat{x}_{e,t+h}^{(0.5)}, \quad \hat{x}_{e,t+h}^{(0.9)},$$

which correspond to the median and the upper decision quantile. The median supports conservative scale-in, while the upper quantile supports earlier scale-out under uncertainty.

Demand representation.

The forecasting formulation is shared across all benchmarks, but the physical meaning of $x_{e,t}$ depends on the entity type. For aggregate and machine-level experiments, $x_{e,t}$ is the observed CPU-utilization signal after one-minute aggregation. For service-level experiments, $x_{e,t}$ is a CPU-core demand proxy obtained by aggregating container-level rows that belong to the same `app_du` and summing

$$\text{cpu_request_cores} \times \text{cpu_utilization}.$$

This keeps the forecasting route consistent while allowing the control layer to operate on service-relevant load units.

Required-capacity mapping.

The controller acts on a required-capacity signal rather than on the raw forecast. For aggregate and machine-level experiments, the observed demand is first normalized to the policy capacity space and then converted into a minimum required capacity

$$c_{e,t}^* = \max((1 + \rho)\tilde{x}_{e,t}, c_{\min}),$$

where ρ is the headroom ratio, $\tilde{x}_{e,t}$ is normalized demand, and $c_{\min}$ is the minimum capacity floor.

For service-level experiments, the same logic is expressed in replica-equivalent units. The required replica-equivalent capacity becomes

$$c_{e,t}^* = \max\left(\frac{(1+\rho)x_{e,t}}{r_e}, c_{\min}\right),$$

where r_e is the median per-container requested CPU cores for service e . This mapping is important because it places reactive, lagged-tracking, and predictive service policies in the same decision space.

Reactive baseline.

The reactive policy is threshold-based. If the current utilization exceeds an upper threshold, capacity is increased by at most one step. If utilization falls below a lower threshold, capacity is decreased by at most one step. This approximates the structure of common threshold-driven elasticity controllers and serves as the operational baseline.

Lagged target-tracking baseline.

The second policy baseline is forecast-free but more explicit than threshold control. It drives the next action directly toward the previous-step required capacity:

$$c_{e,t+1} = \max\left(c_{\min}, c_{e,t} + \text{clip}(c_{e,t}^* - c_{e,t}, -\Delta, \Delta)\right).$$

This policy uses the same minimum-capacity floor and maximum single-step change as the predictive controller, but it has no access to future uncertainty and therefore tests whether naive target chasing is already sufficient.

Predictive policy.

The predictive controller consumes the two quantile forecasts after mapping them into the same required-capacity space as the reactive and lagged baselines. Scale-out decisions use the upper quantile together with an explicit scale-out margin, while scale-in decisions use the median together with a safety margin and a cooldown constraint:

$$\begin{aligned} c_{e,t+1} &= \min(c_{e,t} + \Delta, \hat{c}_{e,t+h}^{(0.9)}), & \text{if scale-out,} \\ c_{e,t+1} &= \max(c_{e,t} - \Delta, (1+\gamma)\hat{c}_{e,t+h}^{(0.5)}), & \text{if scale-in,} \\ c_{e,t+1} &= c_{e,t}, & \text{otherwise.} \end{aligned}$$

where Δ is the maximum single-step change and γ is the scale-in safety margin. Cooldown suppresses oscillatory scale-in without delaying urgent scale-out.

Forecasting backbone.

The proposed model family is framed as an uncertainty-aware multi-scale temporal forecaster. In the present experiments, that interface is instantiated by **UA-MSTCN-Lite**, a paper-specific lightweight quantile-forest surrogate that emits $P50$ and $P90$ forecasts without introducing a heavyweight deep-learning dependency stack. **UA-MSTCN-Lite** is not the full deep UA-MSTCN architecture; it is the reproducible surrogate used to exercise the forecasting-to-control contract in this revision. The surrounding dataset contract, policy simulator, and evaluation protocol are kept stable so that a deeper temporal backbone can replace this surrogate in later work without changing the rest of the study design.

Concretely, **UA-MSTCN-Lite** uses a 60-minute history window and predicts the 1-, 5-, and 10-minute horizons. Its feature extractor includes lag values at 1, 2, 3, 5, 10, 15, 20, 30, 45, and 60 minutes; multi-scale windows of 3, 5, 10, 20, 30, and 60 minutes; per-window mean, standard deviation, minimum, maximum, median, 0.1 quantile, 0.9 quantile, endpoint difference, last-minus-mean, and slope; and additional short/long moving-average and volatility differences. Each horizon is fitted with a separate **RandomForestRegressor** using 200 trees, `min_samples_leaf=3`, `max_features="sqrt"`, random seed $42 + h$, and `n_jobs=-1`. Per-tree predictions define the empirical $P50$ and $P90$ outputs. The $P90$ path is widened by the 0.99 quantile of positive training residuals relative to $P50$, while $P50$ and $P90$ are clipped to the training target support, with the upper path allowed up to 1.10 times the training maximum before the calibration floor. Multi-entity transfer experiments use the same implementation with pooled windows from the training entities.

Calibration objective.

Because the controller consumes quantiles rather than only point forecasts, the evaluation also

measures empirical coverage:

$$\text{cov}_{e,h}^{(q)} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left(x_{e,t_i+h} \leq \hat{x}_{e,t_i+h}^{(q)} \right),$$

where $q \in \{0.5, 0.9\}$. Ideally, $P50$ and $P90$ coverage should remain close to 0.5 and 0.9, respectively. Interval width $\hat{x}_{e,t+h}^{(0.9)} - \hat{x}_{e,t+h}^{(0.5)}$ is tracked alongside coverage because overly wide intervals may protect SLA at the cost of resource efficiency.

Optimization targets.

Forecasting quality is evaluated with MAE, RMSE, MAPE, and pinball-oriented quantile diagnostics. Control quality is evaluated with SLA violation rate, over-provisioning, under-provisioning, scaling-action count, and average reserved capacity. This two-layer evaluation is central to the paper because lower forecast error alone does not imply better autoscaling decisions.

4 Experimental Setup

This section describes the trace-processing and evaluation protocol. The routes move from aggregate cluster usage to machine entities, cross-machine transfer, and an audited service cohort. All forecasting routes use chronological splits, with test windows held out from fitting and calibration.

4.1 Dataset and preprocessing

The experiments use Alibaba `cluster-trace-v2018`, which contains approximately 4000 machines over 8 days and provides machine-level utilization, container-level usage, and batch-workload tables [8]. The study derives an aggregate series from `machine_usage`, a high-coverage machine cohort from the same archive, and a service cohort from `container_meta` plus a full local mirror of `container_usage.tar.gz`.

The preprocessing script reads machine-usage CSV records from the tar archive, drops invalid rows, aggregates by timestamp, and emits one-minute mean CPU utilization, mean memory utilization, and active-machine count. This keeps the aggregate route auditable, while the machine and service routes narrow the construct-validity gap. The service route uses a verified full mirror of `container_usage.tar.gz`, removing the earlier `low-container_id` prefix heuristic.

4.2 High-coverage machine cohort

The machine route tests whether aggregate conclusions survive a finer unit of analysis. It scans `machine_usage.tar.gz`, selects the 12 machines with the highest row counts, aggregates each to one-minute means, and reindexes them onto the common global span. Missing minutes are marked and filled only after selection. The retained machines cover 93.7%–94.7% of the span, with a 5.8% average imputation ratio.

This cohort is not a service-level benchmark; it is a controlled cross-entity check between the aggregate and service routes.

4.3 Container service cohort

The service route extracts application-level evidence from the official container tables. It uses `container_meta.tar.gz` for container-to-`app_du` membership and request sizes, then maps a full mirrored pass over `container_usage.tar.gz` back to services. Service demand is aggregated to one-minute resolution using mean utilization and a CPU-core proxy defined by requested cores times observed CPU utilization.

This is therefore a much broader official-data extension, although it still should not be read as a full population census because the meta-data prefilter intentionally excludes smaller and shorter-lived services (see Table 2). The fixed rule requires at least 60 containers and 650,000 seconds of metadata span, then retains profiled `app_du` groups with matched-container hit ratio at least 0.90 and observed-minute coverage at least 0.96. The final cohort contains 139 services, 19,419 selected containers, 19,269 matched containers, and a common 11,520-minute span; the top-10 overlap audit retains all earlier targeted services.

4.4 Forecasting and policy configuration

The forecasting benchmark implements persistence, moving average, linear regression, random forest, and a feed-forward MLP regressor. It also includes `UA-MSTCN-Lite`, a trainable uncertainty-aware surrogate using multi-scale history features and a quantile-aware tree ensemble. These models validate the harness and define performance floors for later deep temporal variants.

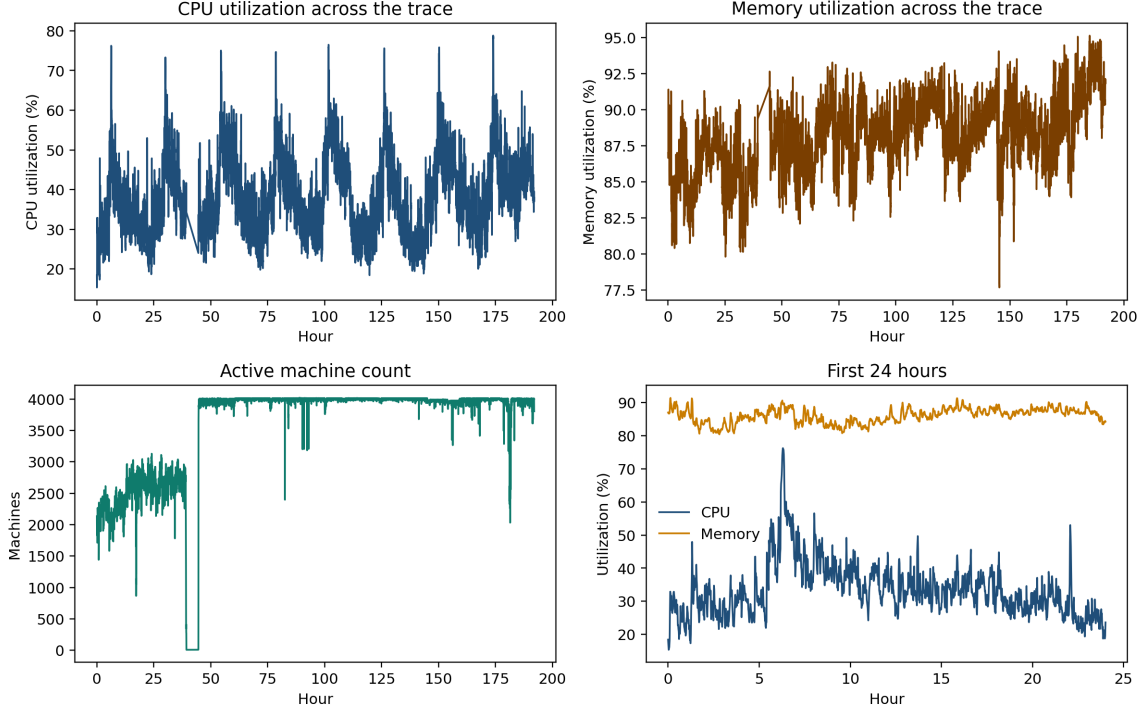


Fig. 1 Aggregate workload overview across the processed Alibaba v2018 series, including (top-left) CPU utilization, (top-right) memory utilization, (bottom-left) active-machine counts, and (bottom-right) first-day zoom.

Table 2 Summary statistics of the broad Alibaba service-level cohort extracted from the official container tables.

Item	Value
Services (app_du)	139
Selected containers	19,419
Matched containers in usage stream	19,269
Common minute span	11,520
Mean observed-minute coverage	0.973
Coverage range	0.972–0.973
Mean container hit ratio	0.993
Mean imputation ratio	0.027

Table 3 Statistics of the processed aggregate Alibaba v2018 workload series.

Item	Value
Observed one-minute samples	11,208
Minute-index span	0–11,519
Mean CPU utilization	37.996%
Peak CPU utilization	78.788%
Mean memory utilization	88.007%
Median active machine count	3994.917

The revision adds quantile linear regression (QLR) as a simpler uncertainty-aware baseline on the aggregate route. QLR emits $P50$ and raw $P90$ forecasts for 1-, 5-, and 10-minute horizons; artifacts retain raw $P90$, raw crossing counts/rates, and guarded $\max(P90_{\text{raw}}, P50)$ outputs. QLR fitting uses only the train-plus-validation prefix, and the test split is used only for evaluation. Service-level QLR pilot and calibration artifacts remain bounded audit evidence, not a full 139-service result.

The aggregate series uses a chronological 70/15/15 split. The first 85% supports training and validation; the final 15% is held out for test evaluation. The default context window is 60 minutes, horizons are 1, 5, and 10 minutes, and forecasting quality is measured with MAE, RMSE, MAPE, and pinball loss.

The autoscaling simulator compares a reactive threshold policy, a lagged target-tracking policy, and a predictive policy using median and high-quantile forecasts with cooldown and step-size constraints. All policies act in the same required-capacity space; the predictive route uses

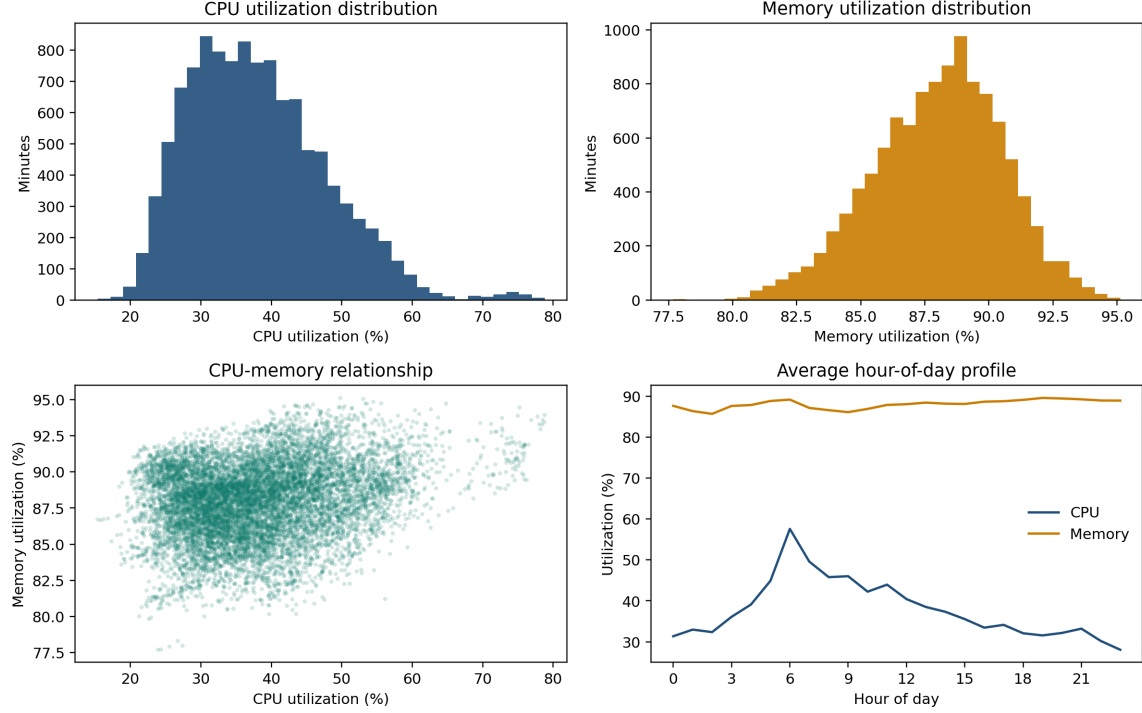


Fig. 2 Additional views of the processed series: CPU distribution, memory distribution, CPU-memory relationship, and average hour-of-day profile.

the calibrated upper quantile plus the configured scale-out margin.

4.5 Extended analysis protocol

Beyond the core tables, the protocol tests context sensitivity (30, 60, and 120 minutes), 5-minute regime behavior, empirical quantile calibration, policy ablations, cooldown/margin sensitivity, 3-fold cross-machine zero-shot transfer, and service-level forecasting plus control. For service policy, the resampling unit is `service`. The primary statistic is the equal-weighted mean predictive-minus-reactive delta; a load-weighted mean using the service CPU-cores proxy is auxiliary. Stability is assessed with 10,000 paired bootstrap resamples over services and seed 20260311: intervals fully below zero indicate improvement, intervals fully above zero indicate degradation, and intervals crossing zero are not stable. The same pre-fixed rule governs whether the broad audited cohort replaces earlier targeted evidence.

The broad service route also computes mean load, coefficient of variation, 95th-percentile-to-mean burstiness, lag-1 autocorrelation,

imputation ratio, and usable observed-minute count. Rank-based median splits report policy deltas, and a compact taxonomy groups services into SLA-improved/over-improved, SLA-improved/actions-worsened, over-improved/SLA-worsened, all-worsened, near-neutral, and other mixed cases. Near-neutral requires all three absolute policy deltas to fall below the cohort-specific first-quartile absolute-delta threshold. The machine cohort reuses the same leakage-free setup for direct comparison.

4.6 Artifacts and environment

The pipeline writes outputs to `paper-workbench/experiments/results/`. The metrics contract is fixed by `metrics.csv`, policy series by `policy_series.csv`, and figures are rendered from artifacts rather than edited manually.

The experiments used TeX Live 2023, `pdflatex`, `latexmk`, and a Python environment with `pandas`, `scikit-learn`, `pyyaml`, and `matplotlib`.

Top-12 service CPU demand traces within the selected `app_du` cohort

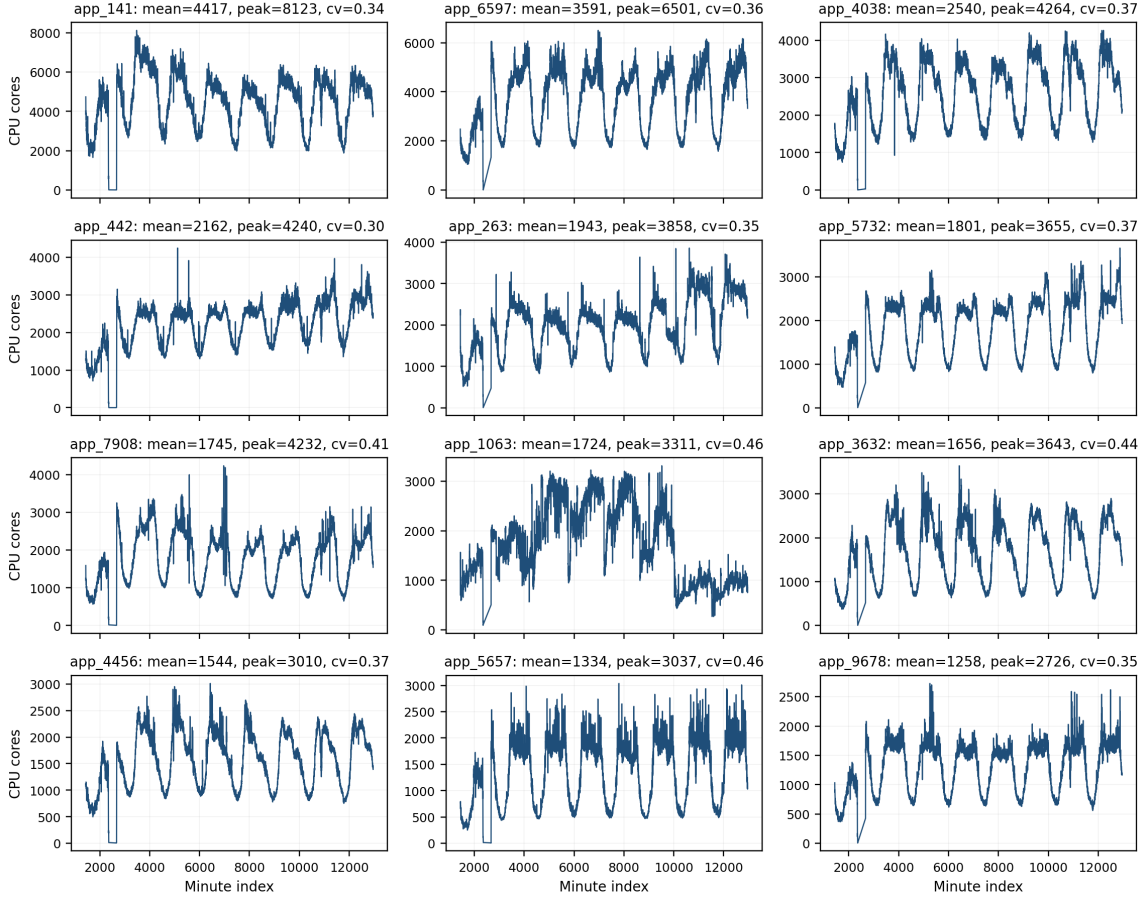


Fig. 3 Twelve representative high-demand services from the broad `app_du` cohort. Even under the same shared minute span, the retained services differ markedly in mean load, smoothness, and burstiness.

5 Results and Discussion

The results report leakage-free forecasting, calibration, transfer, service-level evidence, and control outcomes. They support a boundary-mapping claim rather than a universal model-improvement claim.

5.1 Forecasting benchmark

Table 4 summarizes aggregate forecasting on 11,208 held-out samples. Linear regression has the best average MAE (3.514), random forest is close (3.529), and UA-MSTCN-Lite remains competitive rather than dominant (3.548). The MLP improves over simple heuristics but does not change the ranking. At the 10-minute horizon, linear regression and UA-MSTCN-Lite differ by only

0.0013 MAE, so the aggregate result supports using the surrogate for control while rejecting a point-forecast superiority claim.

Table 5 adds the reviewer-requested QLR baseline, fitted only on the train-plus-validation prefix and evaluated only on the test segment. QLR is competitive at the 1- and 5-minute horizons, but it is used as an auditable uncertainty baseline rather than a new superiority claim. Raw and guarded $P90$ outputs are retained, with zero raw and guarded crossings in the aggregate run.

5.2 Case study and accuracy-runtime trade-off

Figure 5 plots aligned test slices for the 1-, 5-, and 10-minute horizons. Persistence lags at turns, the MLP smooths turning points, and random forest

Table 4 Leakage-free forecasting performance on the aggregate cluster series (MAE, RMSE, MAPE).

Model	Horizon	MAE	RMSE	MAPE
Persistence	1	2.395	3.285	0.060
Moving average	1	3.215	4.348	0.079
Linear regression	1	2.268	3.123	0.056
Random forest	1	2.324	3.176	0.058
MLP regressor	1	2.422	3.267	0.061
UA-MSTCN-Lite	1	2.349	3.214	0.058
Persistence	5	4.569	6.162	0.112
Moving average	5	4.387	5.850	0.107
Linear regression	5	3.967	5.324	0.098
Random forest	5	3.928	5.237	0.097
MLP regressor	5	4.276	5.603	0.105
UA-MSTCN-Lite	5	3.986	5.304	0.098
Persistence	10	5.119	6.785	0.126
Moving average	10	4.845	6.409	0.119
Linear regression	10	4.307	5.765	0.107
Random forest	10	4.334	5.780	0.107
MLP regressor	10	4.427	5.781	0.109
UA-MSTCN-Lite	10	4.308	5.793	0.107

Table 5 Aggregate quantile linear regression baseline on the held-out test split.

Model	H	MAE	RMSE	P90 cov.	Raw crosses
QLR	1	2.265	3.145	0.900	0
QLR	5	3.967	5.390	0.883	0
QLR	10	4.315	5.861	0.886	0

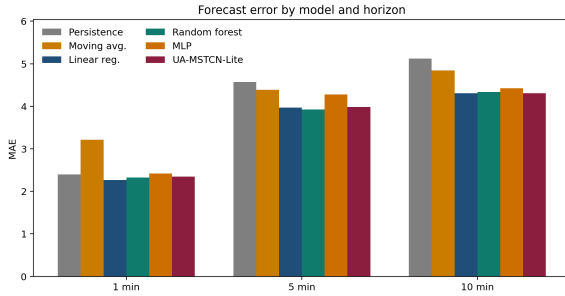


Fig. 4 Forecast error by model and horizon after the leakage-free split correction. Linear regression remains the strongest average point forecaster, while the standard neural MLP baseline improves over simple heuristics without changing the overall ranking.

and UA-MSTCN-Lite track the main shape more closely. The figure remains relevant because the controller consumes the surrogate’s quantile band, not only its median.

Figure 6 compares average MAE with runtime. Linear regression is strongest and fastest among the main point baselines at 19.3 ms per

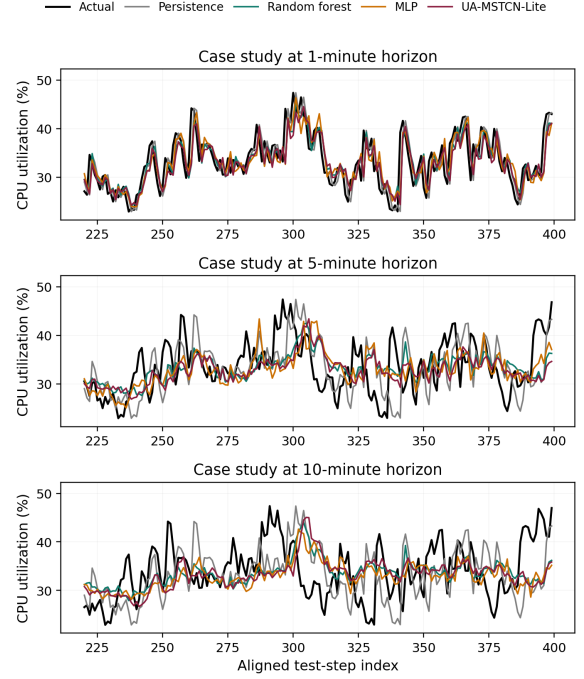


Fig. 5 Forecast case study on aligned test slices for the 1-, 5-, and 10-minute horizons, with persistence, linear regression, random forest, MLP regressor, and UA-MSTCN-Lite predictions overlaid.

call; the MLP takes about 630 ms; random forest is slightly more accurate than MLP but much slower. UA-MSTCN-Lite stays close to random forest in MAE while adding quantile outputs, separating point accuracy from controller-facing uncertainty.

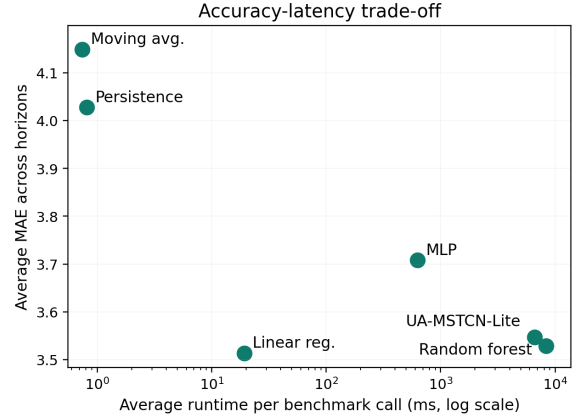


Fig. 6 Average forecasting accuracy versus average runtime per benchmark call on the current workstation.

5.3 Context, regime, and calibration analysis

Figure 7 shows context-window sensitivity. A 60-minute window gives the best average MAE (3.548), compared with 3.615 for 30 minutes and 3.585 for 120 minutes.

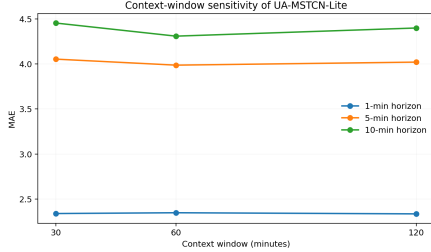


Fig. 7 Context-window sensitivity of UA-MSTCN-Lite. The 60-minute default gives the best average MAE across the three evaluation horizons.

Figure 8 partitions the 5-minute test set by recent volatility. UA-MSTCN-Lite is slightly better than random forest in the moderate regime (3.731 vs. 3.752 MAE), but slightly worse in stable and bursty regimes, identifying bursty demand as the clearest target for a stronger model.

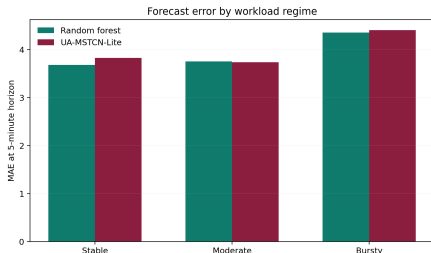


Fig. 8 Forecast error at the 5-minute horizon across stable, moderate, and bursty workload regimes.

Table 6 and Figure 9 evaluate quantile coverage. $P50$ stays close to 0.5, while $P90$ is conservative at 1 minute (0.920) and mildly under-covered at 5 and 10 minutes (0.867 and 0.868). The band is usable for control, but longer-horizon under-coverage justifies the explicit scale-out margin.

5.4 Cross-entity machine-cohort benchmark

The 12-machine cohort tests whether the aggregate story survives a finer unit of analysis. Table 7

Table 6 Empirical quantile coverage of UA-MSTCN-Lite on the aggregate test split.

Horizon	$P50$ cov.	$P90$ cov.	Mean int. width
1 minute	0.506	0.920	4.623
5 minutes	0.516	0.867	6.709
10 minutes	0.500	0.868	7.405

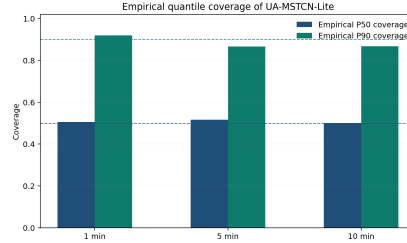


Fig. 9 Empirical $P50$ and $P90$ coverage of UA-MSTCN-Lite. Dashed lines indicate the nominal target levels of 0.5 ($P50$) and 0.9 ($P90$).

shows the same direction, but more strongly: linear regression is the best point predictor at every horizon (MAE 3.633, 5.520, and 5.966), the MLP is second, and UA-MSTCN-Lite trails both MLP and random forest on this cohort.

Table 7 Average forecasting performance across the 12-machine high-coverage cohort.

Model	H	MAE	RMSE	MAPE
Persistence	1	3.716	5.185	0.103
Linear regression	1	3.633	4.926	0.101
MLP regressor	1	3.921	5.246	0.109
Random forest	1	4.012	5.396	0.107
UA-MSTCN-Lite	1	4.121	5.647	0.106
Persistence	5	6.268	8.497	0.172
Linear regression	5	5.520	7.318	0.153
MLP regressor	5	5.845	7.683	0.159
Random forest	5	6.170	8.223	0.162
UA-MSTCN-Lite	5	6.244	8.423	0.161
Persistence	10	6.932	9.390	0.192
Linear regression	10	5.966	7.959	0.165
MLP regressor	10	6.301	8.324	0.172
Random forest	10	6.548	8.745	0.173
UA-MSTCN-Lite	10	6.651	8.936	0.172

Figure 10 shows the same pattern by entity. Linear regression wins 6, 8, and 8 of the 12 machines at the 1-, 5-, and 10-minute horizons; UA-MSTCN-Lite wins 2, 3, and 2. The current surrogate is therefore not the strongest general-purpose point forecaster at finer granularity.

Figure 11 shows that most of the gap comes from a subset of machines. The worst case is `m_2014`, where UA-MSTCN-Lite reaches 9.37 average MAE versus 6.04 for linear regression.

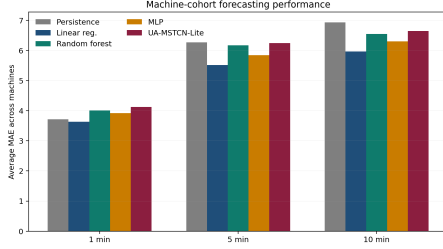


Fig. 10 Average forecasting MAE across the 12-machine high-coverage cohort.

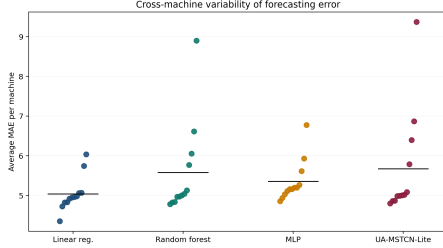


Fig. 11 Per-machine average MAE across the 12-machine cohort. Each point is one machine and the horizontal segment marks the model mean.

The uncertainty result is more stable than the point-ranking result. Table 8 shows average $P50$ coverage near 0.5 and $P90$ coverage between 0.847 and 0.891, so longer-horizon under-coverage is not unique to the aggregate series.

Table 8 Average quantile coverage of UA-MSTCN-Lite across the 12-machine cohort.

Horizon	$P50$ cov.	$P90$ cov.	Mean int. width
1 minute	0.510	0.891	7.529
5 minutes	0.523	0.853	9.375
10 minutes	0.517	0.847	9.825

5.5 Cross-machine transfer

The transfer benchmark trains pooled global models on 8 machines and evaluates zero-shot on 4 held-out machines, repeated over 3 deterministic folds. Table 9 summarizes the result.

Pooled linear regression transfers almost as well as entity-specific linear regression: its mean MAE differs by only -0.004 , -0.017 , and -0.013 at the 1-, 5-, and 10-minute horizons. The global MLP is the next-best zero-shot point predictor,

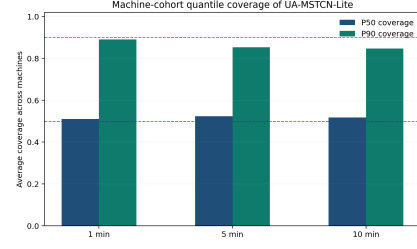


Fig. 12 Average $P50$ and $P90$ coverage of UA-MSTCN-Lite across the 12-machine cohort.

Table 9 Cross-machine zero-shot transfer on held-out machines.

Horizon	Pers.	Glob. lin.	Glob. MLP	Glob. RF	Glob. UA	Ent. lin.
1 minute	3.716	3.629	3.735	4.496	3.934	3.633
5 minutes	6.268	5.503	5.722	6.293	6.328	5.520
10 minutes	6.932	5.953	6.286	6.642	6.648	5.966

but still trails global linear regression at every horizon.

Global UA-MSTCN-Lite does not beat the linear point baseline, but its held-out-machine $P90$ coverage remains 0.904, 0.859, and 0.853. The transfer result therefore preserves the same boundary: linear point forecasts lead, while the surrogate supplies controller-facing uncertainty.

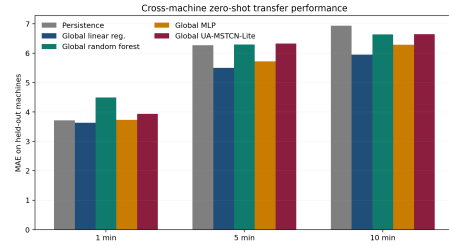


Fig. 13 Cross-machine zero-shot transfer performance across the 3 deterministic folds.

5.6 Container service-level evidence

The service cohort narrows the construct-validity gap by moving from aggregate and machine abstractions to 139 audited `app_du` groups over the shared 11,520-minute span. The overlap audit retains all 10 earlier targeted services, but the broader selection changes the service-level headline: point-forecast leadership shifts to linear regression, and the earlier action-reduction claim no longer survives.

Table 10 shows that linear regression is strongest at every horizon (MAE 19.999, 34.250, and 43.463). Persistence is the overall runner-up, and the MLP remains ahead of UA-MSTCN-Lite but behind both linear regression and persistence. The earlier targeted ranking favoring UA-MSTCN-Lite does not survive broader audited service coverage.

Table 10 Service-level forecasting across the audited 139-service **app.du** cohort.

Model	1 min.	5 min.	10 min.	Avg.
Persistence	20.408	34.638	45.098	33.381
Linear regression	19.999	34.250	43.463	32.571
MLP regressor	23.684	38.395	49.117	37.066
Random forest	26.317	37.728	44.912	36.319
UA-MSTCN-Lite	27.939	39.746	45.677	37.787

Figure 14 shows the same result visually. The broader service route matters because it exposes heterogeneity that a small targeted subset missed.

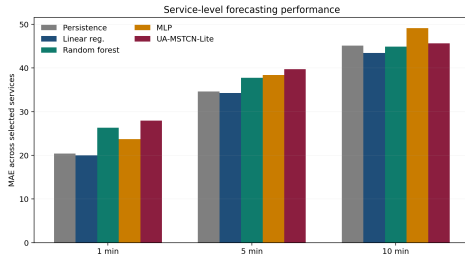


Fig. 14 Average MAE across the audited 139-service **app.du** cohort. Under broader service coverage, point-forecast leadership shifts to linear regression and the generic MLP baseline remains mid-pack rather than dominant.

At service level, UA-MSTCN-Lite still provides usable upper quantiles: P_{90} coverage is 0.921, 0.896, and 0.877 at the three horizons. P_{50} coverage is slightly low (0.469–0.481), but the upper quantile remains close enough for the predictive-controller test.

The service-level control comparison is stricter because demand is heterogeneous. Averaged over 139 services, lagged target tracking lowers over-provisioning to 3.848 but raises SLA violation to 0.486 and actions to 1660.9. Relative to reactive control, the predictive policy raises mean SLA violation from 0.0173 to 0.0311 and actions from

249.0 to 399.4, while mean over-provisioning falls from 56.310 to 53.677.

The paired service bootstrap fixes the headline. Predictive-minus-reactive SLA delta is +0.0138 with 95% interval [0.0035, 0.0271], and scaling-actions delta is +150.4 with interval [114.0, 185.0], so both degradations are stable. Over-provisioning differs: the mean delta is -2.633, but its interval [-6.953, 2.328] crosses zero. The auxiliary load-weighted action delta is nearly zero (-0.567 with interval [-64.257, 68.245]), indicating that the equal-weighted action penalty comes mainly from many smaller services rather than the highest-load services alone.

Figure 15 shows the same heterogeneity: SLA and action deltas stay positive through most service ranks, while over-provisioning straddles zero for many services.

Table 11 Lightweight mechanism summary for predictive-minus-reactive service outcomes on the audited broad cohort. Panel A reports equal-weighted mean deltas after rank-based median splits on selected service descriptors. Panel B reports an outcome taxonomy; near-neutral requires all three absolute deltas to fall below the cohort-specific first-quartile absolute-delta thresholds. Lower is better for all three metrics.

Panel A: Descriptor splits					
Split	Half	Svcs	SLA Δ	Over Δ	Acts Δ
Mean load	High	69	+0.011	-0.800	+45.4
Mean load	Low	70	+0.017	-4.440	+254.0
Burstiness	High	69	+0.023	+4.970	+82.2
Burstiness	Low	70	+0.005	-10.127	+217.7
Lag-1 autocorr.	High	69	+0.026	-5.731	+95.0
Lag-1 autocorr.	Low	70	+0.002	+0.421	+205.1
Imputation ratio	High	69	+0.018	+0.105	+144.7
Imputation ratio	Low	70	+0.010	-5.332	+156.1

Panel B: Outcome taxonomy					
Category		Svcs	Share (%)	Load share (%)	
SLA improved,	Over	6	4.3		6.5
improved					
SLA improved,	Actions	7	5.0		4.7
worsened					
Over improved,	SLA	87	62.6		47.9
worsened					
All worsened		12	8.6		4.9
Near-neutral		3	2.2		3.0
Other mixed		24	17.3		33.0

Table 11 adds a mechanism view. SLA degradation is strongest in high-burstiness (+0.0227) and high-autocorrelation (+0.0255) services, while over-provisioning gains concentrate in low-burstiness (-10.127) and low-load (-4.440)

halves. The dominant category is “over improved, SLA worsened” (87 of 139 services); only 6 services improve both SLA and over-provisioning. Thus the shared global predictive policy mainly trades lower over-provisioning for worse SLA and actions, with the harshest SLA penalties in burstier and more persistent services.

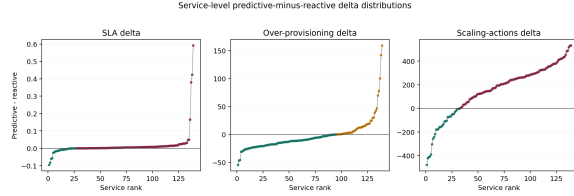


Fig. 15 Sorted predictive-minus-reactive deltas across the audited 139-service cohort. Negative values are favorable for all three metrics. The positive centers of the SLA and scaling-actions panels show that those penalties are widespread rather than caused only by a few extreme services, while over-provisioning remains heterogeneous around zero.

5.7 Trace-driven control comparison

Table 12 summarizes aggregate control. Reactive control achieves near-zero SLA violation by carrying mean capacity 0.821 against average required capacity 0.470. Lagged target tracking nearly eliminates over-provisioning but incurs 47.1% SLA violation and 1616 actions. The predictive policy sits between them: SLA violation rises to 3.46%, but over-provisioning falls by 59.8% relative to reactive control, with mean capacity 0.610. All policies share the same normalized required-capacity space.

Table 12 Reactive, lagged-target, and predictive capacity control on the 5-minute evaluation horizon.

Policy	SLA	Over	Under	Acts	Cap.
Reactive threshold	0.001	0.351	0.000	43	0.821
Lagged target tracking	0.471	0.013	0.014	1616	0.469
Predictive	0.035	0.141	0.001	589	0.610

Using QLR as the predictive uncertainty source gives the same aggregate boundary: SLA violation 0.034, over-provisioning 0.147, and 587 actions at the 5-minute horizon. QLR is safer than

lagged target tracking but worse than reactive control on SLA and actions, and it is not treated as a full 139-service result.

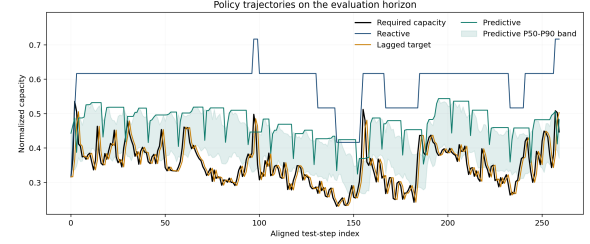


Fig. 16 Reactive, lagged-target, and predictive capacity trajectories on the 5-minute evaluation horizon. The lagged baseline shows the failure mode of naive target chasing, while the predictive policy follows the calibrated high-quantile forecast with far lower SLA risk.

5.8 Policy ablation and sensitivity

Table 13 shows that the controller depends mainly on the scale-out margin and cooldown. Removing the margin raises SLA violation from 0.0346 to 0.1137; removing cooldown raises actions from 589 to 1617 and SLA violation to 0.1119. A median-only policy is unsafe (0.3337 SLA violation), while removing the scale-in guard has almost no effect.

Table 13 Predictive-policy ablation on the 5-minute evaluation route.

Variant	SLA	Over	Under	Acts	Cap.
Full predictive policy	0.035	0.141	0.001	589	0.610
No scale-out margin	0.114	0.089	0.004	589	0.555
No cooldown	0.112	0.100	0.005	1617	0.565
Median only	0.334	0.035	0.017	359	0.488
No scale-in guard	0.035	0.141	0.001	589	0.610

Figure 17 sweeps scale-out margin and cooldown. The default ($m = 0.10, c = 8$) is not optimal on a single axis, but it sits near the middle of the observed Pareto surface: smaller margins reduce over-provisioning but raise SLA risk, while larger margins and longer cooldown consume more steady capacity.

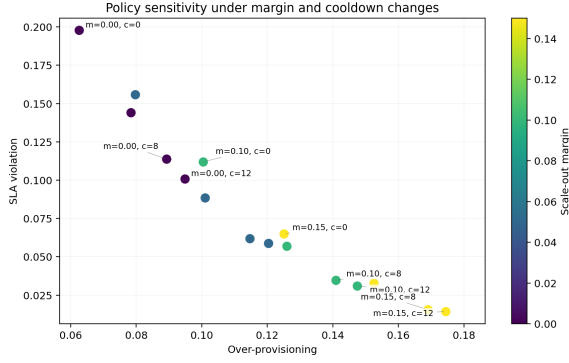


Fig. 17 Sensitivity of the predictive controller to scale-out margin and cooldown. The selected default operates in the middle of the observed SLA-versus-over-provisioning trade-off.

The final boundary is narrow: the simulator provides reproducible trace-level evidence about when predictive control helps or fails, but it does not prove that the same policy transfers unchanged to a live cluster controller.

6 Threats to Validity

Several validity threats remain.

External validity.

The study uses public Alibaba traces rather than a live platform, so the conclusions are strongest for trace-driven comparison and weaker for direct deployment claims.

Construct validity.

The service cohort narrows the construct-validity gap but is not a full service census. It is bounded by a metadata prefilter requiring at least 60 containers and 650,000 seconds of span, followed by usage-profile and post-aggregation quality thresholds. The benchmark should therefore be read as a broad rule-based service cohort, not the full app.du population.

Internal validity.

UA-MSTCN-Lite is a lightweight trainable surrogate, not the final deep UA-MSTCN architecture. The reported numbers therefore apply to the verified surrogate and policy route, not to an unimplemented deeper model.

Conclusion validity.

Forecasting gains do not automatically imply better capacity control, so forecasting and policy metrics are kept separate. The service-descriptor splits and taxonomy are descriptive benchmark analyses, not causal identification for live services.

7 Conclusion

This paper contributes a reproducible public-trace benchmark for predictive autoscaling across aggregate, machine-level, transfer, and broad service-level evidence. Linear regression remains the strongest point-forecast baseline across the main benchmarks, while UA-MSTCN-Lite contributes calibrated upper quantiles for predictive control.

The central contribution is therefore boundary mapping rather than a new controller architecture. Aggregate predictive control reduces over-provisioning relative to reactive control, but the audited 139-service cohort shows higher mean SLA violation and scaling actions, with no stable over-provisioning gain under paired service bootstrap. Descriptor splits locate the harshest SLA penalties in burstier and more persistent services and the clearest over-provisioning relief in lower-load and lower-burstiness halves.

Next steps are service-aware tuning under held-out-service evaluation and a stronger temporal backbone tested against the same linear and persistence baselines.

References

- [1] Lorigo-Botran, T., Miguel-Alonso, J., Lozano, J.A.: A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of Grid Computing* **12**(4), 559–592 (2014)
- [2] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J.: Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade. *Queue* **14**(1), 70–93 (2016) <https://doi.org/10.1145/2898442.2898444>
- [3] Bernstein, D.: Containers and cloud: From lxc to docker to kubernetes. *IEEE Cloud Computing* **1**(3), 81–84 (2014) <https://doi.org/10.1109/MCC.2014.51>

- [4] Dinda, P.A., O'Hallaron, D.R.: Host load prediction using linear models. *Cluster Computing* **3**(4), 265–280 (2000) <https://doi.org/10.1023/A:1019048724544>
- [5] Casalicchio, E.: A study on performance measures for auto-scaling cpu-intensive containerized applications. *Cluster Computing* **22**(3), 995–1006 (2019) <https://doi.org/10.1007/s10586-018-02890-1>
- [6] Rossi, A., Visentin, A., Carraro, D., Prestwich, S., Brown, K.N.: Forecasting workload in cloud computing: towards uncertainty-aware predictions and transfer learning. *Cluster Computing* **28**(4) (2025) <https://doi.org/10.1007/s10586-024-04933-2>
- [7] Alibaba Cluster Trace Program: Alibaba Cluster Trace Program. <https://github.com/alibaba/clusterdata>. Accessed 2026-03-10 (2026)
- [8] Alibaba Cluster Trace Program: Cluster Trace v2018 Documentation. https://raw.githubusercontent.com/alibaba/clusterdata/master/cluster-trace-v2018/trace_2018.md. Accessed 2026-03-10 (2026)
- [9] Jiang, C., Qiu, Y., Shi, W., Ge, Z., Wang, J., Chen, S., Cerin, C., Ren, Z., Xu, G., Lin, J.: Characterizing co-located workloads in alibaba cloud datacenters. *IEEE Transactions on Cloud Computing* **10**(4), 2381–2397 (2022) <https://doi.org/10.1109/TCC.2020.3034500>
- [10] Guo, J., Chang, Z., Wang, S., Ding, H., Feng, Y., Mao, L., Bao, Y.: Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In: 2019 IEEE/ACM International Symposium on Quality of Service (2019)
- [11] Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., Xu, C.: Characterizing microservice dependency and performance: Alibaba trace analysis. In: *Proceedings of the ACM Symposium on Cloud Computing*, pp. 412–426 (2021)
- [12] Zargar Azad, B., Torres, A.M., Demchenko, Y., Laat, C.: A cloud-iot framework for efficient resource management of microservice applications. *Journal of Grid Computing* **21**(2), 28 (2023)
- [13] Puebla, G., Gil, R., Lent, R.: Cdascaler: a cost-effective dynamic autoscaling approach for containerized microservices. *Soft Computing* (2024)
- [14] Zanussi, L., Tessera, D., Massari, L., Bermejo, B., Juiz, C., Calzarossa, M.: Load-aware predictive auto-scaling framework for cloud environments. *Cluster Computing* **29**, 197 (2026) <https://doi.org/10.1007/s10586-026-05944-x>
- [15] Reiss, C., Tumanov, A., Ganger, G.R., Katz, R., Kozuch, M.A.: Google Cluster-Usage Traces: Format + Schema. <https://github.com/google/cluster-data/blob/master/ClusterData2011.2.md> (2012)
- [16] Canizares, P.C., Nunez, A., Bernal, A., Cambronero, M.E., Barker, A.: Simcan2cloud: a discrete-event-based simulator for modelling and simulating cloud computing infrastructures. *Journal of Cloud Computing* **12**(1), 133 (2023) <https://doi.org/10.1186/s13677-023-00511-w>
- [17] Islam, S., Keung, J., Lee, K., Liu, A.: Empirical prediction models for adaptive resource provisioning in the cloud. *Future Generation Computer Systems* **28**(1), 155–162 (2012)
- [18] Calheiros, R.N., Masoumi, E., Ranjan, R., Buyya, R.: Workload prediction using arima model and its impact on cloud applications' qos. *IEEE Transactions on Cloud Computing* **3**(4), 449–458 (2015) <https://doi.org/10.1109/TCC.2014.2350475>
- [19] Herbst, N.R., Huber, N., Kounev, S., Amrehn, E.: Self-adaptive workload classification and forecasting for proactive resource provisioning. *Concurrency and Computation: Practice and Experience* **26**(12), 2053–2078 (2014) <https://doi.org/10.1002/cpe.3224>
- [20] Baldan, F.J., Ramirez-Gallego, S., Bergmeir,

- C., Herrera, F., Benitez, J.M.: A forecasting methodology for workload forecasting in cloud systems. *IEEE Transactions on Cloud Computing* **6**(4), 929–941 (2018) <https://doi.org/10.1109/TCC.2016.2586064>
- [21] Banerjee, S., Roy, S., Khatua, S.: Efficient resource utilization using multi-step-ahead workload prediction technique in cloud. *The Journal of Supercomputing* **77**(9), 10636–10663 (2021) <https://doi.org/10.1007/s11227-021-03701-y>
- [22] Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997)
- [23] Song, B., Yu, Y., Zhou, Y., Wang, Z., Du, S.: Host load prediction with long short-term memory in cloud computing. *The Journal of Supercomputing* **74**(12), 6554–6568 (2017) <https://doi.org/10.1007/s11227-017-2044-4>
- [24] Bi, J., Li, S., Yuan, H., Zhou, M.: Integrated deep learning method for workload and resource prediction in cloud systems. *Neurocomputing* **424**, 35–48 (2021) <https://doi.org/10.1016/j.neucom.2020.11.011>
- [25] Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T.: Deepar: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* **36**(3), 1181–1191 (2020)
- [26] Lim, B., Arik, S.O., Loeff, N., Pfister, T.: Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting* **37**(4), 1748–1764 (2021)
- [27] Mao, M., Li, J., Humphrey, M.: Cloud auto-scaling with deadline and budget constraints. In: 2010 11th IEEE/ACM International Conference on Grid Computing, pp. 41–48 (2010). <https://doi.org/10.1109/GRID.2010.5697966> . <https://doi.org/10.1109/GRID.2010.5697966>
- [28] Kim, W.-Y., Lee, J.-S., Huh, E.-N.: Study on proactive auto scaling for instance through the prediction of network traffic on the container environment. In: Proceedings of the 11th International Conference on Ubiquitous Information Management and Communication, pp. 1–8 (2017). <https://doi.org/10.1145/3022227.3022243> . <https://doi.org/10.1145/3022227.3022243>
- [29] Al-Dhuraibi, Y., Paraiso, F., Djarallah, N., Merle, P.: Autonomic vertical elasticity of docker containers with elasticdocker. In: 2017 IEEE 10th International Conference on Cloud Computing (CLOUD), pp. 472–479 (2017). <https://doi.org/10.1109/CLOUD.2017.67> . <https://doi.org/10.1109/CLOUD.2017.67>
- [30] Gul, O.M.: Heuristic resource reservation policies for public clouds in the iot era. *Sensors* **22**(23), 9034 (2022) <https://doi.org/10.3390/s22239034>
- [31] Minarolli, D., Mazrekaj, A., Freisleben, B.: Tackling uncertainty in long-term predictions for host overload and underload detection in cloud computing. *Journal of Cloud Computing* **6**(1) (2017) <https://doi.org/10.1186/s13677-017-0074-3>
- [32] Weiss, K., Khoshgoftaar, T.M., Wang, D.: A survey of transfer learning. *Journal of Big Data* **3**(1) (2016) <https://doi.org/10.1186/s40537-016-0043-6>
- [33] Ye, R., Dai, Q.: A novel transfer learning framework for time series forecasting. *Knowledge-Based Systems* **156**, 74–99 (2018) <https://doi.org/10.1016/j.knosys.2018.05.021>
- [34] Liu, C., Jiao, J., Li, W., Wang, J., Zhang, J.: Tr-predictor: An ensemble transfer learning model for small-sample cloud workload prediction. *Entropy* **24**(12), 1770 (2022) <https://doi.org/10.3390/e24121770>
- [35] Hao, J., Yue, K., Zhang, B., Duan, L., Fu, X.: Transfer learning of bayesian network for measuring qos of virtual machines. *Applied Intelligence* **51**(12), 8641–8660 (2021) <https://doi.org/10.1007/s10489-021-02362-x>

Statements and Declarations

Funding

The authors received no specific funding for this study.

Competing interests

The authors declare no competing interests.

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The raw Alibaba Cluster Trace Program datasets are publicly available from the Alibaba Cluster Trace Program repository. The reproducibility repository at <https://github.com/Jinchun-Liu/paper-workbench> provides the manuscript source, preprocessing route, cohort-construction scripts, result-table generation route, and figure-generation artifacts needed to reproduce the reported numbers from the public traces.

Materials availability

Not applicable.

Code availability

All preprocessing scripts, model training code, broad service-cohort construction scripts, and predictive-control simulators used in this study are publicly released at <https://github.com/Jinchun-Liu/paper-workbench>.

Author contribution

Jinchun Liu conceived the study, designed the methodology, implemented the experiments, analyzed the results, and drafted the manuscript. Haoxun Li contributed to manuscript revision and final submission preparation. Both authors reviewed and approved the final manuscript.